

Proplinq V2 API Documentation

This documentation covers the implementation of **Property Search**, **Booking**, **Calendar Management**, and **Live AI Chat WebSockets** for both Web (Next.js) and Mobile (Flutter).

1. Property Search API (Public)

Search supports **hotels**, **shortlets**, and **real estate listings** (apartments). It accepts both:

- **Natural language** (intent-based) queries via `q`
- **Structured filters** (type, location, bedrooms, price range)

Endpoint

Endpoint: `GET /api/v1/properties/search`

Query Parameters

- `q` (string, optional): Natural language query (e.g. "250k shortlet in Lekki")
- `type` (string, optional): One of `apartment`, `hotel`, `shortlet`
- `location` (string, optional): Free-text location filter (partial match)
- `bedrooms` (int, optional): Bedroom count
- `price_min` (number, optional): Minimum price
- `price_max` (number, optional): Maximum price
- `per_page` (int, optional): Defaults to 12
- `page` (int, optional)

Natural Language Interpretation (Supported Patterns)

When `q` is provided, the API attempts to infer structured filters:

- **Property type:** "shortlet", "short let", "hotel", "real estate", "apartment", "house", "flat"
- **Bedrooms:** "2 bed", "2 bedroom", "two bedroom"
- **Price:** "250k", "250,000", "250000 naira", "under 500k", "200k-300k"
- **Location:** Parses "in <location>" (e.g. "shortlet in Lekki", "apartment in Victoria Island")

If the user does not specify a type, the API returns results across all property types.

Ranking (Relevance)

Results are ranked using a relevance score that favors:

- Matches on inferred **type** and **location**
- Matches in `title`, `location`, and `description`
- Listings with images and room data

Examples

1) "shortlet in Lekki" `GET /api/v1/properties/search?q=shortlet%20in%20Lekki`

2) "hotel in Ikeja" `GET /api/v1/properties/search?q=hotel%20in%20Ikeja`

3) "2 bedroom apartment in Victoria Island" `GET /api/v1/properties/search?q=2%20bedroom%20apartment%20in%20Victoria%20Island`

4) "250k shortlet in Lekki" `GET /api/v1/properties/search?q=250k%20shortlet%20in%20Lekki`

5) Combined filters + keywords GET /api/v1/properties/search?type=shortlet&location=Lekki&price_max=250000&q=near%20beach

Response (Success - 200)

```
{
  "status": true,
  "message": "Success",
  "data": {
    "data": [
      {
        "id": 123,
        "type": "shortlet",
        "title": "Modern 2 Bedroom Shortlet",
        "location": "Lekki, Lagos",
        "price": "250000.00"
      }
    ],
    "current_page": 1,
    "per_page": 12,
    "total": 1
  }
}
```

2. Booking API

Guests use this API to reserve properties (Apartments, Hotels, or Shortlets).

Optimization Note: Bookings are processed against a real-time availability engine. For high-traffic events, ensure your UI handles the 409 Conflict status gracefully when rooms sell out instantly.

How it works:

When a guest attempts to book, the system performs a real-time availability check against both **confirmed bookings** and **manual host blocks**. If the dates are free, a pending booking is created, and a payment URL is returned for the guest to complete the transaction.

Create a Booking

Endpoint: POST /api/v1/bookings

Authentication: Required (Bearer Token)

Request Body:

```
{
  "property_id": 1,
  "check_in_date": "2026-04-10",
  "check_out_date": "2026-04-15"
}
```

Response (Success - 201):

```
{
  "success": true,
  "data": {
    "booking": {
      "uuid": "...",
      "booking_code": "PLQ-ABC123",
      "amount": 50000.00,
      "status": "pending"
    },
    "payment": {
      "authorization_url": "https://checkout.paystack.com/..."
    }
  }
}
```

3. Calendar Management (Agent/Host)

Agents can manage availability by manually blocking or unblocking dates.

This is **room-based**, meaning the calendar endpoints work on `room_id` (not `property_id`). Hotels and shortlets both support calendar management as long as the listing has rooms.

Why use this?

Hosts may need to take a room off the market for maintenance, personal use, or offline bookings. By "blocking" a date, the host ensures that no guest can book the property through the app for those specific days.

Authentication

Required (Bearer Token). User must be an `agent`, and must own the property that the room belongs to.

Get Room Calendar (Agent)

Endpoint: GET `/api/v1/agent/calendar/room/{room_id}?start_date=2026-04-01&end_date=2026-04-30`

Query Params

- `start_date` (required, date)
- `end_date` (required, date, must be `>= start_date`)

Response (Success - 200)

```
{
  "status": true,
  "message": "Room calendar retrieved successfully.",
  "data": [
    {
      "date": "2026-04-01",
      "status": "available",
      "is_blocked": false,
      "metadata": []
    }
  ]
}
```

Block Dates (Agent)

Endpoint: POST /api/v1/agent/calendar/room/{room_id}/block

Request Body:

```
{
  "start_date": "2026-04-20",
  "end_date": "2026-04-22",
  "status": "blocked",
  "reason": "Maintenance"
}
```

Notes

- status is optional and can be blocked or available (defaults to blocked)
- start_date must be today or later

Unblock Dates (Agent)

Endpoint: DELETE /api/v1/agent/calendar/unblock/{uuid}

4. Public Availability Check

Guests can check if a room is available before booking.

Endpoint: GET /api/v1/rooms/{room_id}/availability?start_date=2026-04-01&end_date=2026-04-30

Response:

```
{
  "success": true,
  "data": [
    { "date": "2026-04-01", "status": "available", "is_blocked": false },
    { "date": "2026-04-02", "status": "booked", "is_blocked": true },
    { "date": "2026-04-03", "status": "blocked", "is_blocked": true, "metadata": { "reason": "Maintenance" } }
  ]
}
```

5. AI Chat WebSockets (Live Updates)

The AI Chat uses **Laravel Reverb** (Pusher-compatible) for real-time response streaming.

Why WebSockets?

Instead of making the user wait for a standard HTTP response (which can take 5-10 seconds for AI), the app can immediately acknowledge the query and "listen" for the AI's response to be pushed live. This provides a much smoother, chat-like experience.

Next.js Implementation

1. **Install:** `npm install laravel-echo pusher-js`

Security Note: Anonymous AI conversations are restricted by a signed token or user session logic. Ensure the `conversationId` is stored securely in the client state and never exposed in public logs.

2. Setup:

```
import Echo from 'laravel-echo';
import Pusher from 'pusher-js';

const echo = new Echo({
  broadcaster: 'reverb',
  key: 'YOUR_REVERB_KEY',
  wsHost: 'api.proplinq.com',
  wsPort: 8080,
  forceTLS: false,
  enabledTransports: ['ws', 'wss'],
});

// Subscribe to conversation
echo.channel(`ai-chat.${conversationId}`)
  .listen('ai.chat.response', (data) => {
    console.log("AI Message:", data.response);
  });
```

Flutter Implementation

1. **Dependency:** `pusher_channels_flutter: ^2.2.0`

2. Setup:

```
import 'package:pusher_channels_flutter/pusher_channels_flutter.dart';

PusherChannelsFlutter pusher = PusherChannelsFlutter.getInstance();

await pusher.init(
  apiKey: "YOUR_REVERB_KEY",
  host: "api.proplinq.com",
  port: 8080,
  onEvent: (event) {
    if (event.eventName == 'ai.chat.response') {
      print("AI Response: ${event.data}");
    }
  },
);

await pusher.subscribe(channelName: "ai-chat.${conversationId}");
await pusher.connect();
```

6. User Verification (Email & Phone)

All users must verify their identity via OTP (One-Time Password).

Registration Flow

Upon successful registration (POST /api/v1/register), the user's phone_verified_at is set to null by default. An OTP is automatically sent to the user's phone via SMS/WhatsApp.

Phone Verification

Endpoint: POST /api/v1/verify-phone (or POST /api/v1/verify-otp in AuthController)

Authentication: Required (Bearer Token)

Request Body:

```
{
  "code": "123456"
}
```

Effect: Updates phone_verified_at to the current timestamp.

Email Verification

Endpoint: POST /api/v1/verify-email

Authentication: Required (Bearer Token)

Request Body:

```
{
  "code": "123456"
}
```

Effect: Updates email_verified_at to the current timestamp.

Resend Codes

- **Resend Email OTP:** POST /api/v1/resend-verification-code
- **Resend Phone OTP:** POST /api/v1/resend-otp

Developer Notes

- **Free WebSocket:** We use Laravel Reverb, which is self-hosted and free (no Pusher fees).
- **Manual Availability:** If availability_status is false for a room, it means the agent hasn't set any manual availability data yet.
- **AI Engine:** All AI responses are processed in a dedicated Docker container (ai-engine) and broadcasted immediately to the user's channel.

Structural & Response Standard

- **Unified Responses:** All API endpoints follow a strict format:

```
{
  "status": true,
  "message": "Success message",
  "data": { ... }
}
```

- **Error Handling:** Standardized 4xx and 5xx error responses are returned using the same structure with status: false.
- **WebSocket Scalability:** Reverb is configured for easy horizontal scaling using a Redis backplane if traffic increases.

7. Deep Linking (Mobile & Web)

Proplinq supports deep links to allow users to navigate directly to specific property listings within the mobile application.

Deep Link Format (Mobile)

Mobile applications should listen for the following URL scheme to open property details directly.

Format: `proplinq://{property_type}/{property_id}`

Examples:

- `proplinq://apartment/123`
- `proplinq://hotel/456`
- `proplinq://shortlet/789`

Shareable Web Links

When sharing a property on social media or via messaging, use the web-based shareable URL. This URL will automatically attempt to open the Proplinq app on the user's device, or fallback to the App Store/Play Store if the app is not installed.

Format: `https://proplinq.com/property/{property_type}/{property_id}`

How it works:

1. User clicks the link.
2. The browser opens a landing page that attempts to trigger the `proplinq://` deep link.
3. If the app is installed, it opens immediately to the property.
4. If the app is not installed, the landing page displays a fallback option to download the app from the relevant store (iOS/Android).

Dynamic Meta Tags (SEO & Social Sharing)

The shareable web links generate dynamic OpenGraph (OG) and Twitter meta tags, ensuring that when a link is shared on platforms like WhatsApp, Twitter, or Facebook, it displays:

- **Title:** The property name.
- **Description:** A short summary of the property.
- **Image:** The primary image of the property listing.